Charotar University of Science and Technology
Chandubhai S. Patel Institute of Technology
V. T. Patel Department of Electronics and Communication Engineering

FPGA Implementation of an Optimal FIR Filter Using the Parks-McClellan Algorithm

Course: EC317 - Modeling Using Verilog

Bhaumik Soni, 23EC134

Internal Supervisor: Dr. Arpita Patel

1. Abstract

This project presents the design and FPGA implementation of a 21-tap low-pass Finite Impulse Response (FIR) filter optimized using the Parks–McClellan (Remez Exchange) algorithm. The objective is to smooth a square-wave signal by removing its high-frequency harmonic components. The filter coefficients are generated, scaled to fixed-point format, and implemented in Verilog HDL. The complete design—consisting of a test signal generator, shift-register-based FIR structure, and multiply–accumulate (MAC) operation—is synthesized on a Spartan-6 XC6SLX9 FPGA. Real-time filtering performance is verified using a Digital Storage Oscilloscope, where the filtered output shows significantly smoother waveform characteristics compared to the raw input. The results confirm successful low-pass filtering and demonstrate how DSP algorithms can be efficiently realized on FPGA hardware for real-time applications.

2. Introduction

Digital signal processing (DSP) is an essential part of modern electronic systems, enabling tasks such as noise removal, signal smoothing, data compression, and spectrum shaping. One of the most widely used DSP operations is filtering, where specific frequency components of a signal are either preserved or attenuated. Among different types of digital filters, Finite Impulse Response (FIR) filters are preferred in many real-time applications because they are inherently stable, have a predictable linear-phase response, and are highly suitable for parallel hardware implementation on FPGAs.

Many practical signals, such as square waves, contain a fundamental low-frequency component along with many high-frequency harmonics. These harmonics create sharp edges, noise, and distortion in the signal. To obtain a cleaner and smoother signal, a low-pass filter is required to pass only the low-frequency content and suppress the higher-frequency harmonics. FIR filters perform this task accurately when designed with proper coefficients.

In this project, a 21-tap optimal low-pass FIR filter is designed using the Parks–McClellan (Remez Exchange) algorithm, which minimizes the maximum error in the frequency response and produces an equiripple characteristic. The filter coefficients are generated, scaled to 16-bit fixed-point format, and implemented using Verilog HDL. The design is synthesized and tested on a Spartan-6 XC6SLX9 FPGA.

To verify the filter's performance in hardware, an internal square-wave test signal is generated using a frequency divider. This test signal is given as the input to the FIR filter, and both the input and the filtered output are observed using a Digital Storage Oscilloscope (DSO). The filtered waveform appears smoother with reduced harmonic content, demonstrating successful low-pass filtering in real time.

This project shows how theoretical DSP algorithms can be translated into efficient FPGA hardware. It also highlights the advantages of FPGA-based filters, such as high processing speed, parallel computation, and real-time signal handling. The work demonstrates a complete flow—from coefficient generation to Verilog implementation, synthesis, pin assignment, and oscilloscope-based validation—making it a practical example of digital filter implementation on FPGA

3. Objective

The main objective of this project is to design, implement, and verify a 21-tap low-pass FIR filter on an FPGA using Verilog HDL. The project aims to:

1. Design an optimal low-pass FIR filter

- Use the Parks–McClellan (Remez Exchange) algorithm to generate equiripple, minimum-error filter coefficients.
- Convert the coefficient values into 16-bit fixed-point format suitable for hardware implementation.

2. Implement the FIR filter in Verilog

3. Generate a test signal on FPGA hardware

- Use an internal frequency divider to produce a low-frequency square wave.
- Provide a clear input signal to visualize filtering performance.

4. Deploy the complete system on a Spartan-6 FPGA

- Synthesize, implement, and download the Verilog design to the XC6SLX9 FPGA.
- Assign proper I/O pins using a UCF file for clock, switch, and oscilloscope output.

5. Validate filtering results using a Digital Storage Oscilloscope

- Compare the raw square-wave input and the filter output.
- Demonstrate reduction of high-frequency harmonics and smoothing of the waveform.

6. Show the practical capability of FPGA-based DSP

- Prove that FPGAs can perform real-time filtering efficiently.
- Highlight the advantages of parallel hardware implementation for DSP applications.

4. Methodology

4.1 Filter Coefficient Generation

The 21-tap low-pass FIR filter coefficients were generated using the Parks–McClellan (Remez Exchange) algorithm. MATLAB/Python was used to design an equiripple low-pass response. The floating-point coefficients were scaled to 16-bit signed integers so they can be implemented on FPGA using fixed-point arithmetic. These coefficients were stored directly inside the FIR module using an `initial` block.

4.2 FIR Filter Architecture and Code

The FIR filter uses a **shift register + multiply-accumulate (MAC)** architecture. The most recent 21 input samples are stored in an array and shifted every clock. Each sample is multiplied with a coefficient, and the sum of all products is produced as the output.

FIR Filter Code

```
module parks_mcclellan_fir #(parameter N = 21)(
    input clk,
    input signed [15:0] x_in,
    output reg signed [31:0] y_out
);
    reg signed [15:0] x_reg [0:N-1];
    reg signed [15:0] h [0:N-1];
    integer i;

    initial begin
        h[0]  = -1244; h[1]  = -276;  h[2]  = 677;   h[3]  = 1288;
h[4]  = 549;
        h[5]  = -1246; h[6]  = -2241; h[7]  = -447;  h[8]  = 4228;
h[9]  = 9332;
        h[10] = 11546; h[11] = 9332;  h[12] = 4228;  h[13] = -447;
h[14] = -2241;
        h[15] = -1246; h[16] = 549;   h[17] = 1288;  h[18] = 677;
h[19] = -276;
        h[20] = -1244;
    end

    always @(posedge clk) begin
        for (i = N-1; i > 0; i = i - 1)
            x_reg[i] <= x_reg[i-1];
        x_reg[0] <= x_in;

        y_out <= 0;
        for (i = 0; i < N; i = i + 1)
            y_out <= y_out + x_reg[i] * h[i];
    end
endmodule
```

Explanation

- `x_reg[]` stores last 21 samples.
- Every clock, samples shift → implements convolution.

- Each `x_reg[i] * h[i]` performs multiplication.
- Accumulation forms final convolution output.
- Fully real-time at 50 MHz.

4.3 Test Signal Generator and Frequency Divider

A test signal must be provided to verify the FIR filter. The FPGA clock is 50 MHz, which is too fast to observe filtering effect on an oscilloscope. Therefore, a frequency divider is used to slow down the signal.

Test Signal Generator Code

```
module test_signal_gen(
    input clk,
    output reg signed [15:0] test_signal
);
    reg [23:0] counter = 0;

    always @(posedge clk) begin
        counter <= counter + 1;
        if (counter[23])
            test_signal <= 16'sd1000;
        else
            test_signal <= -16'sd1000;
    end
endmodule
```

Why this frequency divider is needed

1. Input square wave must be slow
   The oscilloscope cannot clearly show filtering on a 50 MHz square wave.
   Using `counter[23]` divides frequency by 2^24:

   50 MHz224≈3 Hz\frac{50\,\text{MHz}}{2^{24}} \approx 3\,\text{Hz}22450MHz ≈3Hz

   A slow square wave makes filtering visible.

2. Allows clear before/after comparison
   - Input: sharp edges
   - Output: smooth waveform
     Filtering effect becomes obvious.
3. Removes need for an external function generator
   FPGA internally generates the test signal.

4.4 Top-Level Module Integration

All modules are connected inside a single top-level file. A switch selects whether to output the filtered signal or the raw FPGA clock.

Top-Level Code

```
module fir_demo_top(
    input CLK50M,
    input sw0,
    output DSO_OUT
);
    wire signed [15:0] test_signal;
    wire signed [31:0] filtered_signal;

    test_signal_gen gen_inst (
        .clk(CLK50M),
        .test_signal(test_signal)
    );

    parks_mcclellan_fir fir_inst (
        .clk(CLK50M),
        .x_in(test_signal),
        .y_out(filtered_signal)
    );

    wire output_bit;
    assign output_bit = sw0 ? filtered_signal[10] : CLK50M;

    assign DSO_OUT = output_bit;
endmodule
```

Explanation

- The test signal is given to FIR filter.
- The filtered 32-bit output is down-scaled by selecting bit-10.
- `sw0 = 1` → oscilloscope shows filtered signal.
- `sw0 = 0` → oscilloscope shows raw clock for comparison.
- Output routed to DSO pin.

4.5 UCF Pin Mapping

The UCF file assigns FPGA pins for clock, switch, and output.

UCF Code

```
NET "*" IOSTANDARD = LVCMOS33;
CONFIG VCCAUX = "3.3";

NET "CLK50M" LOC = "p84";
NET "CLK50M" TNM_NET = "osc";
TIMESPEC "TS_osc" = PERIOD "osc" 20.000ns;

NET "sw0" LOC = "p22";
NET "DSO_OUT" LOC = "p75";
```

Explanation

- Maps signals to physical FPGA pins.
- Defines timing constraint for 50 MHz clock.
- Ensures correct hardware interface for oscilloscope output.

4.6 Hardware Verification

After programming the FPGA, both input and filtered output signals are observed on a DSO. The unfiltered square wave shows sharp transitions and harmonics, while the FIR output appears smooth due to high-frequency attenuation. This confirms correct real-time low-pass filtering.

5. Conclusion

The implementation of the 21-tap low-pass FIR filter using the Parks–McClellan algorithm on the Spartan-6 FPGA successfully demonstrated the practical realization of digital signal processing concepts in hardware. The design accurately filtered the internally generated square wave by removing high-frequency harmonics, resulting in a smooth and rounded output waveform that matched theoretical expectations. The internal frequency divider simplified validation by producing a clear, low-frequency test signal directly on the FPGA, eliminating the need for external generators. Real-time performance at a 50 MHz clock confirmed that fixed-point Verilog implementation of FIR filters is efficient, stable, and highly suitable for FPGA-based DSP applications.

This work forms a strong foundation for more advanced signal-processing systems in the future. The filter can be extended to a higher number of taps to achieve sharper frequency transitions or improved stopband attenuation. The same architecture may also be adapted to design band-pass, high-pass, or multiband filters for communication and audio applications. Further enhancements could include pipelined architectures for higher clock speeds, utilization of FPGA DSP slices for greater efficiency, or implementation of adaptive filtering techniques such as LMS for noise-cancellation and real-time environmental signal processing. Overall, the project demonstrates both the feasibility and scalability of FPGA-based filtering solutions for modern embedded and communication systems.